

RT and ES (CoSc3024)

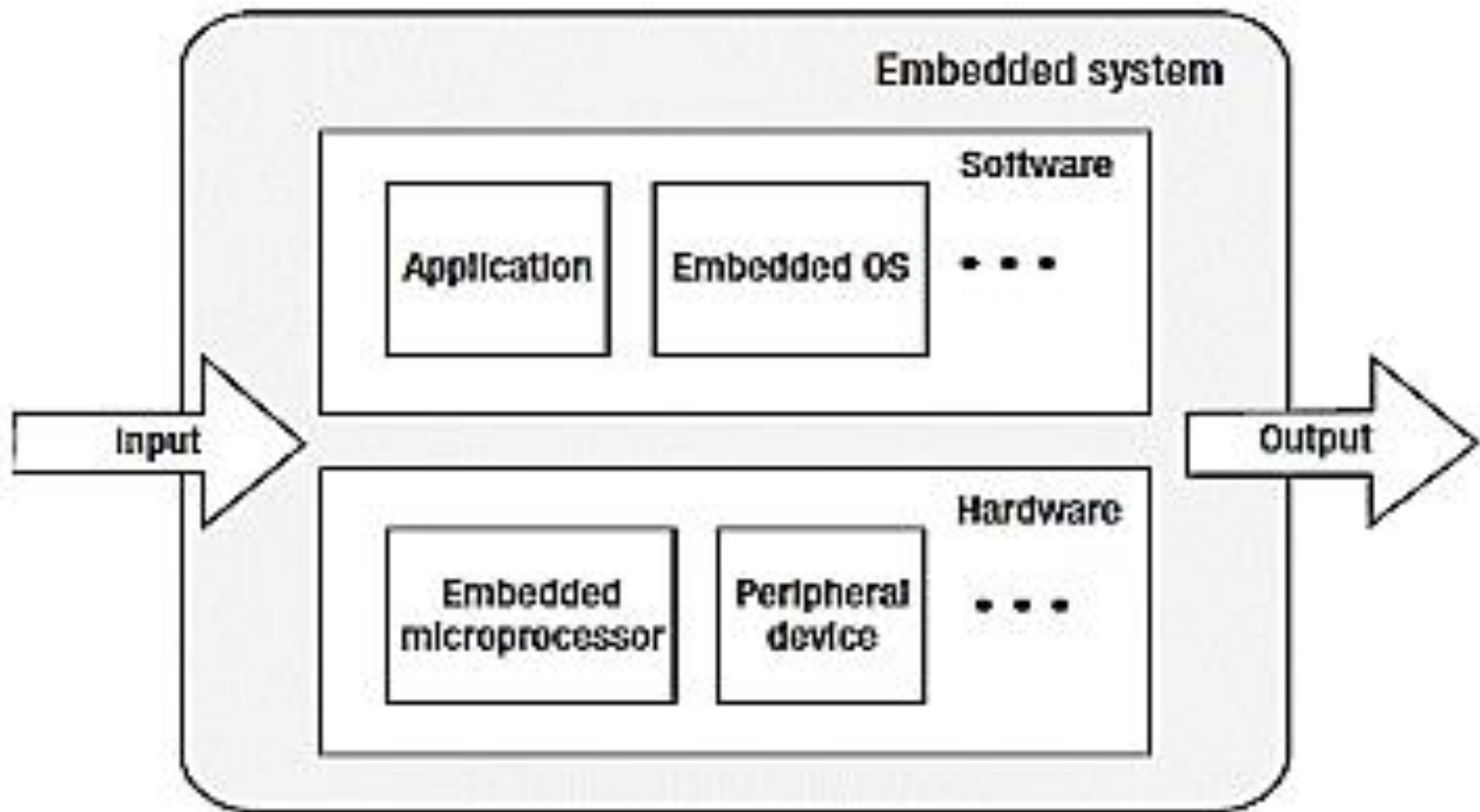
- Chapter 2: Embedded System Architecture
 - Hardware architectures for embedded systems
 - ARM Cortex Mo+ Hardware Overview:
 - Ports, Registers, GPIO, Analog I/O, ADC/DAC
 - Communication:
 - Parallel, USB/Serial, USART, SPI, TWI, Ethernet, Wireless
 - ATmega32 microcontroller Architecture
 - Assembly language Programming with ATmega32 Instruction Set
 - Programming in C to Interface peripherals, Interrupts, ISR and Timers

Ayele S. (Hawassa University IOT)

Hardware architectures for embedded systems

- Embedded hardware is based around *microprocessors* and *microcontrollers*, also include memory, bus, Input/Output, Controller.
- Basically two types of architecture i.e., ***Harvard*** architecture and ***Von Neumann*** architecture are used in embedded systems.
- **Harvard architecture** - when data and code accessed using different buses from different memory blocks.
- **Von Neumann architecture** - data and code accessed using same busses from different memory block.
 - *Read for more detail about two architectures.*

Typical architecture of the Embedded System includes Sensor, Analog to Digital Converter, Memory, Processor, Digital to Analog Converter, Actuators and etc.



Peripheral Devices:

- Embedded system communicates with the outside world via their peripherals, such as:
 - Serial Communication Interfaces (SCI) like RS-232, RS-422, RS-485, etc.
 - Synchronous Serial Communication Interface like I2C, SPI, SSC, and ESSI.
 - Universal Serial Bus (USB)
 - Multi Media Cards (SD Cards, Compact Flash, etc.)
 - And others, ... (read from pdf. file)

Microprocessors:

- Processor is the heart of an embedded system.
- It is the basic unit that takes inputs and produces an output after processing the data.
- For an embedded system designer, it is necessary to have the knowledge of both **microprocessors** and **microcontrollers**.
- A microprocessor is a single VLSI chip having a CPU, which include CU and or ALU.

Microcontroller:

- A microcontroller is a single-chip VLSI unit (also called microcomputer) which,
 - although having limited computational capabilities,
 - possesses enhanced input/output capability and
 - a number of on-chip functional units.

CPU	RAM	ROM
I/O Port	Timer	Serial COM Port

- Microcontrollers are particularly used in embedded systems for real-time control applications with on-chip program memory and devices, (read pdf. file)

ARM Cortex M0+ Hardware Overview

- ARM Cortex-M0+ is a processor, which lays foundation for IOT.
- CAMBRIDGE, UK – March 13, 2012 announced the ARM® Cortex™-M0+ processor, the world's most energy-efficient microprocessor.
- The Cortex-M0+ processor has been optimized to deliver ultra-low-power, low-cost MCUs for d/t appliance.

ARM Cortex M0+ Hardware Overview, ... cont'

- The Cortex-M0+ processor features enable the creation of
 - smart, low-power, microcontrollers to provide efficient communication, management and
 - maintenance across a multitude of wirelessly connected devices, a concept known as the 'Internet of Things'.
- The ARM Cortex-M0+ processor bring 32-bit horsepower to
 - flyweight chips, and it will be suitable for a broad range of industrial and consumer applications.

Ports, Registers, GPIO, Analog I/O, ADC/DAC in embedded system

- **A port** is the point where internal data from MCU chip comes out or external data goes in into MCU.
- They are present in form of PINs of the IC.
- Most of the PINs are dedicated to this function and other pins are used for power supply and others.
- The CPU communicates with these components, reads from them or writes to them.

Ports, Registers, GPIO, Analog I/O, ADC/DAC in embedded system

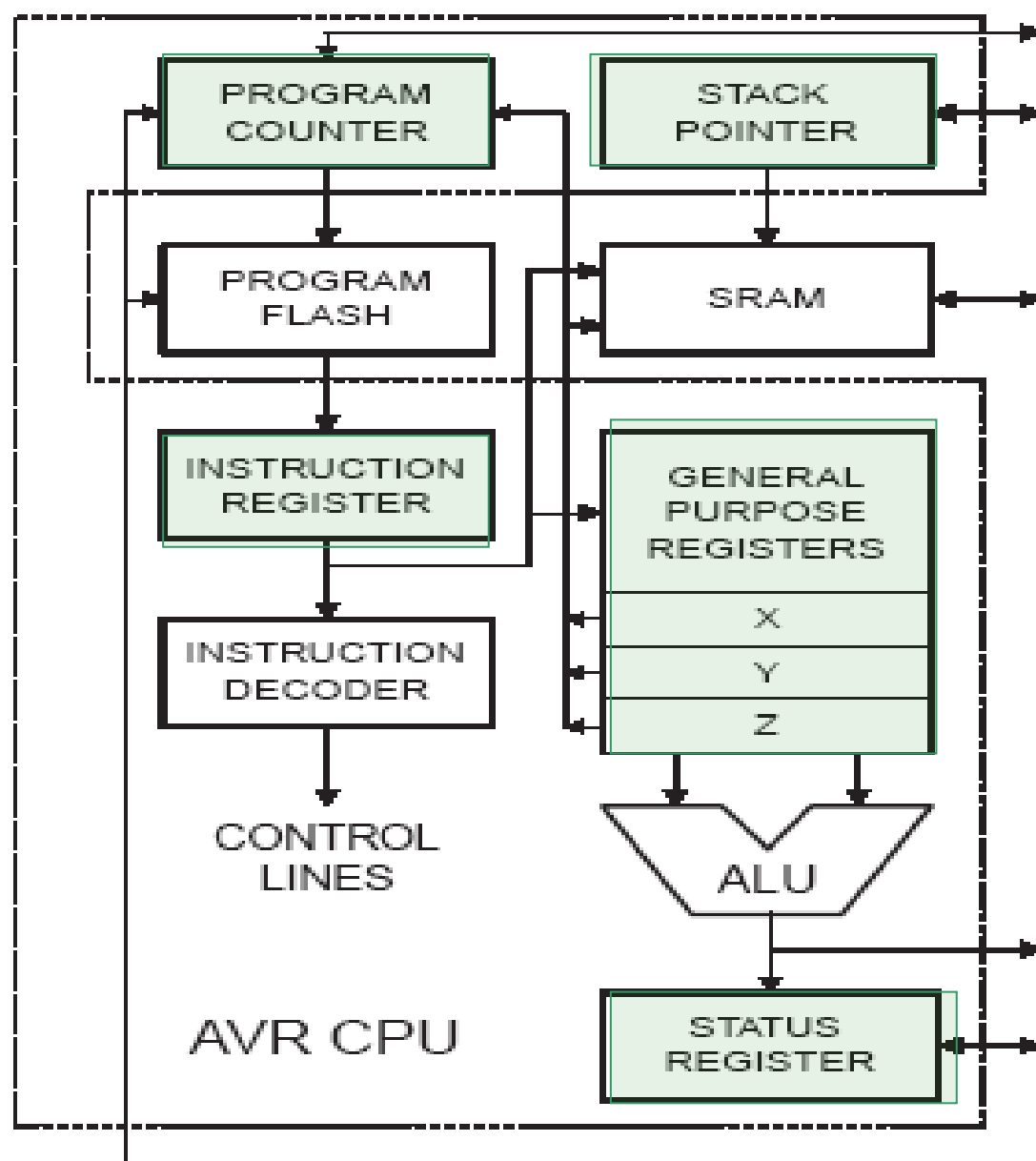
- **Registers** – are a type of computer memory used to quickly accept, store, and transfer data and instructions that being used instantly by the CPU.
- ARM processors provide general-purpose and special-purpose registers.
- Some additional registers are available in privileged execution modes.

Ports, Registers, GPIO, Analog I/O, ADC/DAC in embedded system

- In all ARM processors, the following registers are available and accessible in any processor mode:
 - 13 general-purpose register R0-R12.
 - One Stack Pointer (SP).
 - One Link Register (LR).
 - One Program Counter (PC).
 - One Application Program Status Register (APSR).

Ports, Registers, GPIO, Analog I/O, ADC/DAC in embedded system

- General view of registers



Ports, Registers, GPIO, Analog I/O, ADC/DAC in embedded system

- **GPIO** - (General purpose input/output) is the name of a microcontroller peripheral that provides functionality to source many signals at once (that is, in parallel).
- High-speed GPIO interface connected to the ARM Cortex-M0+ IO bus with up to 29 General-Purpose I/O (GPIO) pins.
- GPIO ports also provide an I/O path for other peripheral functions, such as Timers and Digital-to-Analogue converters.

Analog I/O : Microcontrollers are often required to interface with analog signals.

- They must be able to convert input analog signals, for example from microphone or temperature sensor, to digital data.
- They must also be able to convert digital signals to analog form, for example if driving a loudspeaker or dc motor.

Ports, Registers, GPIO, Analog I/O, ADC/DAC in embedded system

ADC/DAC :

- An analog-to-digital convertor (ADC) is an electronic circuit whose digital output is proportional to its analog input.
- Effectively it "measures" the input voltage, and gives a binary output number proportional to its size.
- Similarly, a digital-to- analog convertor (DAC) is an electronic circuit whose analog output is proportional to its digital input.

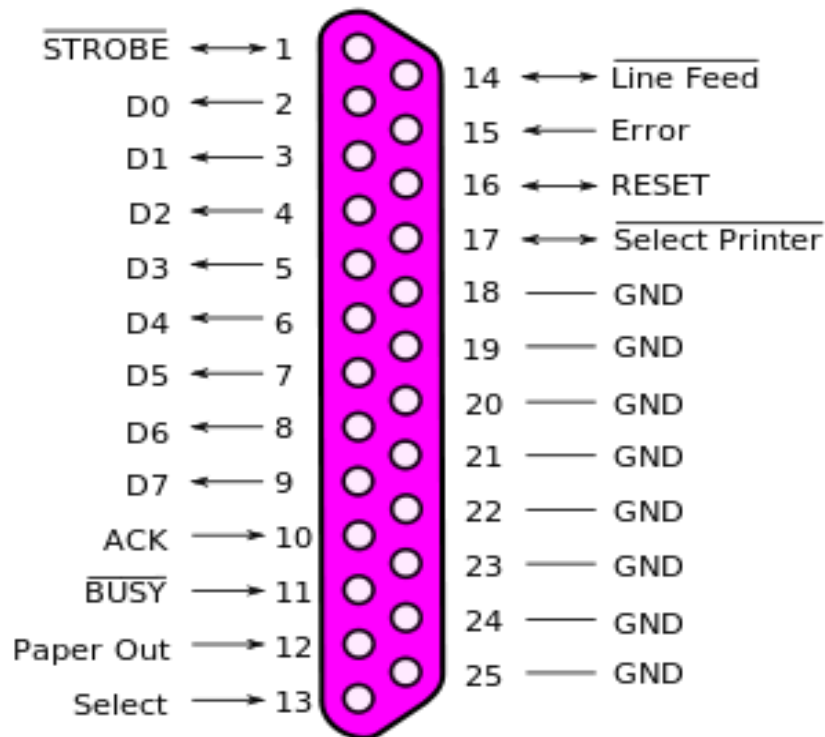
Communication: Parallel, USB/Serial, USART, SPI, TWI, Ethernet

Parallel Communication – is the process of sending/receiving multiple data bits at a time through parallel ports.

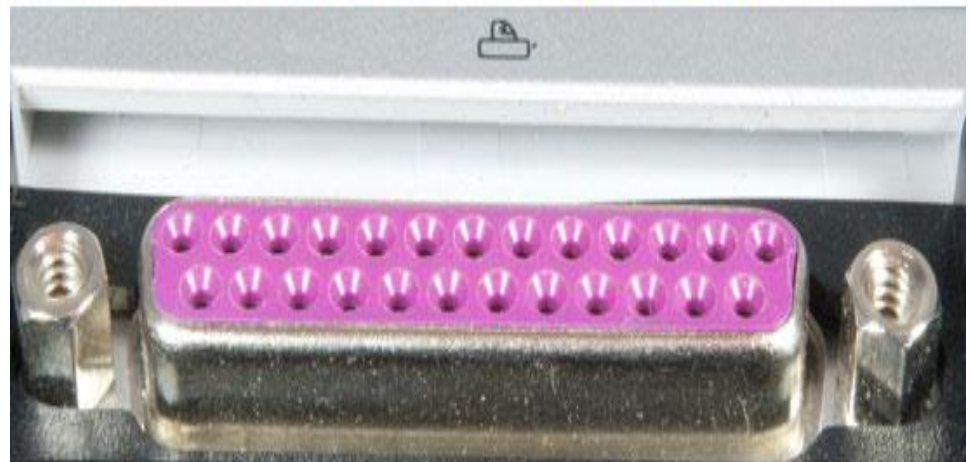
- A parallel port is an interface for a computer to communicate peripherals in parallel manner.
- Data are transferred in or out in parallel, that is, on more than one wire.
- A parallel port carries 1 bit on each wire, thus multiplying the transfer rate obtainable over a single wire.

Communication: Parallel, USB/Serial, USART, SPI, TWI, Ethernet

- The most common type of parallel port is a printer port (e.g., a Centronics port that transfers 8 bits at a time).

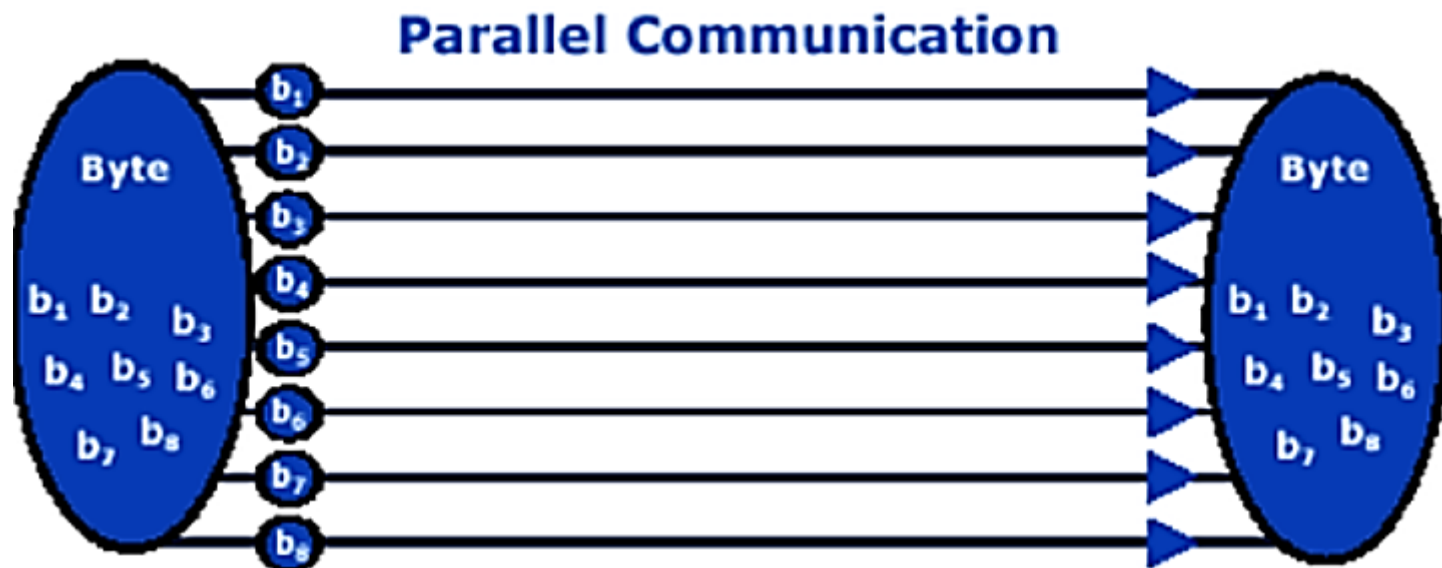


Parallel Port (DB25)



Communication: Parallel, USB/Serial, USART, SPI, TWI, Ethernet

- In parallel communication, all the bits of data are transmitted simultaneously on separate communication lines.
- It is used for shorter distance; more costly than serial communication but faster.



Communication: USB/Serial, USART, SPI, TWI, Ethernet

- **USB/Serial Communication** – is the most popular of all and used for virtually all type of connections.
- USB-bus allows many peripherals to be connected using a single standardized interface socket.
- Data transfer rate in serial communication is measured in terms of bits per second (bps), this is also called as Baud Rate;
- Baud rates of SCI are n-bits per 1200, 2400, 4800, 9600, 19,200 etc. seconds.

Communication: USART, SPI, TWI, Ethernet

USART Communication – Universal Synchronous

Asynchronous Receiver Transmitter (USART) is a kind of serial communication protocols categorized as Synchronous and Asynchronous protocols.

- In synchronous communication, transmission and receiving of data is a continuous stream at a constant rate.
- It requires the clock of transmitting device and receiving device synchronized, and transmission and reception of data occurs with same frequency. Eg. I2C, SPI etc.

Communication: USART, SPI, TWI, Ethernet

USART Communication –

- In asynchronous communication, the transmission of data requires no clock signal.
- Data transfer occurs intermittently rather than steady stream.
- Handshake signals between the transmitter and receiver are important in asynchronous communications.
- **E.g:** Universal Asynchronous Receiver Transmitter (UART), USB, CAN (Controller Area Network), etc.

Communication: SPI, TWI, Ethernet

SPI Communication – Serial Peripheral

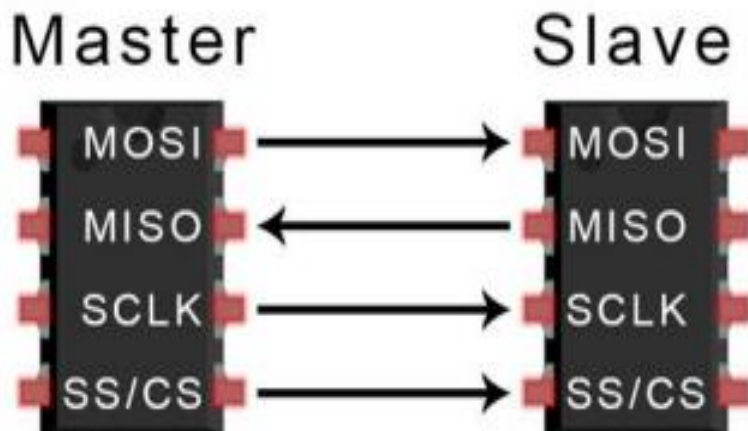
Interface (**SPI**) communication protocol is a common communication protocol used by many different devices.

- One unique benefit of SPI is the fact that data can be transferred without interruption.
- Any number of bits can be sent or received in a continuous stream.
- **E.g:** SD card modules, RFID card reader modules, and 2.4 GHz wireless transmitter/receivers.

Communication: SPI, TWI, Ethernet

SPI Communication –

- Devices communicating via SPI are in a master -slave relationship.
- The master is the controlling device (usually a microcontroller), while the slave (usually a sensor, display, or memory chip) takes instruction from the master.



MOSI (Master Output/Slave Input) – Line for the master to send data to the slave.

MISO (Master Input/Slave Output) – Line for the slave to send data to the master.

SCLK (Clock) – Line for the clock signal.

SS/CS (Slave Select/Chip Select) – Line for the master to select which slave to send data to.

Communication: TWI, Ethernet

TWI Communication –

- The I2C bus has two wires – one for clock, and the other is the data line, which is bi-directional –
- this being the reason it is also sometimes (not always – there are a few conditions) called Two Wire Interface (TWI).
- It is a pretty new and revolutionary technology invented by Philips.

Communication: Ethernet

Ethernet Communication –

- Ethernet has become the networking technology of choice in a wide range of applications;
- Most modern embedded computing products feature at least one Ethernet interface.
- One area that has been reluctant to adopt Ethernet is real-time systems.
- Unpredictable delays in computing systems cannot be tolerated.
- In safety-critical real-time systems, (**where errors can lead to a loss of life**) the requirement for deterministic real-time behavior becomes even more essential.

ATmega32 microcontroller Architecture

- The Atmel®AVR®ATmega32 is a low-power CMOS (compl'tary metal oxide semiconductor) 8-bit microcontroller based on the AVR enhanced RISC architecture.
- By executing powerful instructions in a single clock cycle, the ATmega32 achieves throughputs approaching 1 MIPS per MHz .
- The Atmel®AVR core combines a rich instruction set with 32 general purpose working registers.

The Atmega32 design is based on a Harvard Architecture:

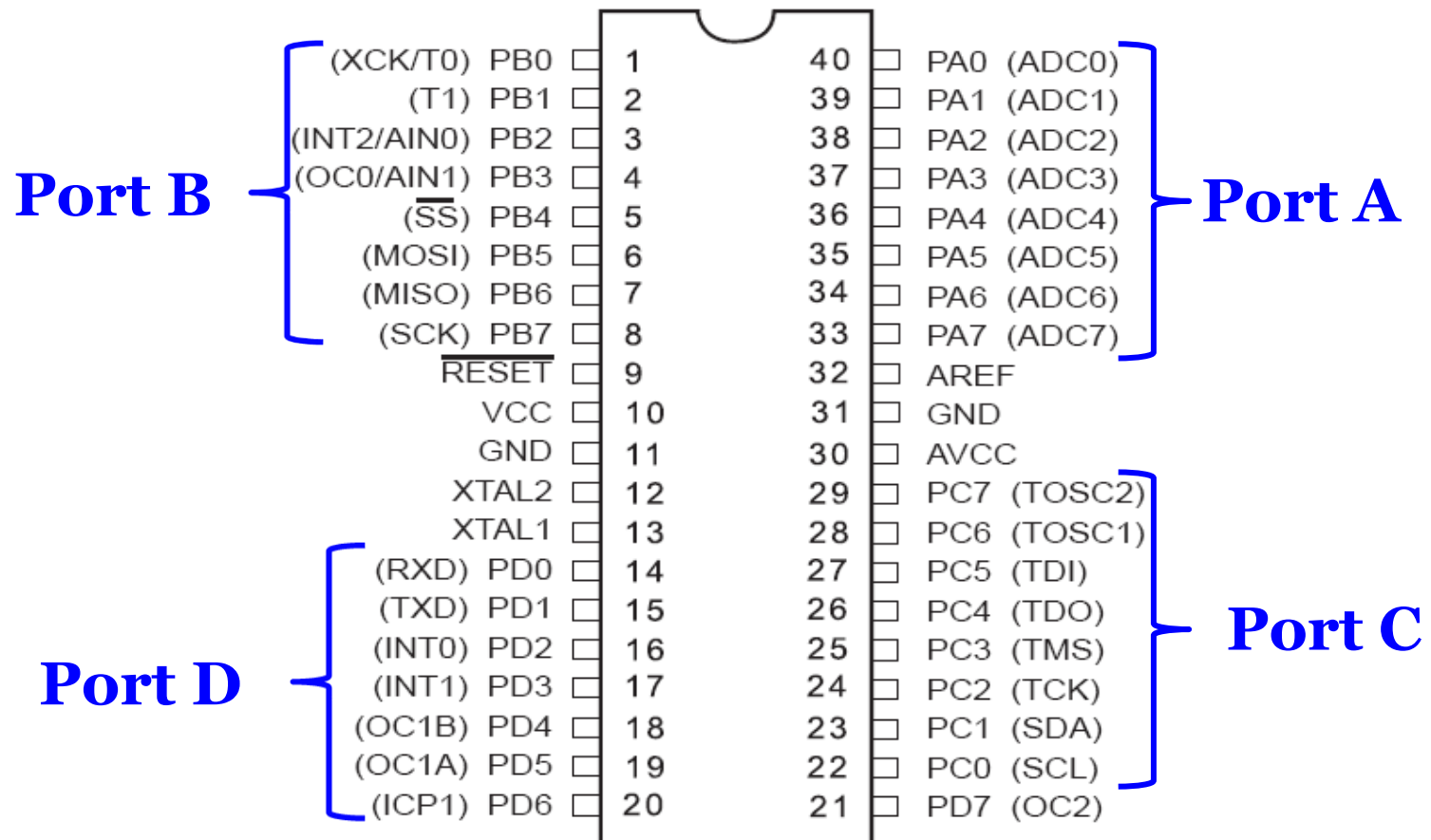
- Assigns data and program instructions to different memory spaces:
 - Program data occupies a different and separate memory from the program itself.
- Each memory space has a separate **bus**, allowing:
 1. Different timing, size, and structure for program instructions and data.
 2. Concurrent access to data and instructions (increases speed).
 3. Clear partitioning of data and instructions (better security).
- **Drawback:** Harder to program.

ATmega32 microcontroller Architecture, ...

- Features of the ATmega32:
 - 32Kbytes of In-System Programmable Flash Program memory with Read-While-Write capabilities.
 - 1Kbytes EEPROM, 2Kbyte SRAM, 32 general purpose I/O lines & 32 general purpose registers.
 - a programmable Watchdog Timer with Internal Oscillator,
 - An SPI port and six software selectable power saving modes, *(Read from pdf. File given)*

ATMega32 Pin out & Descriptions:

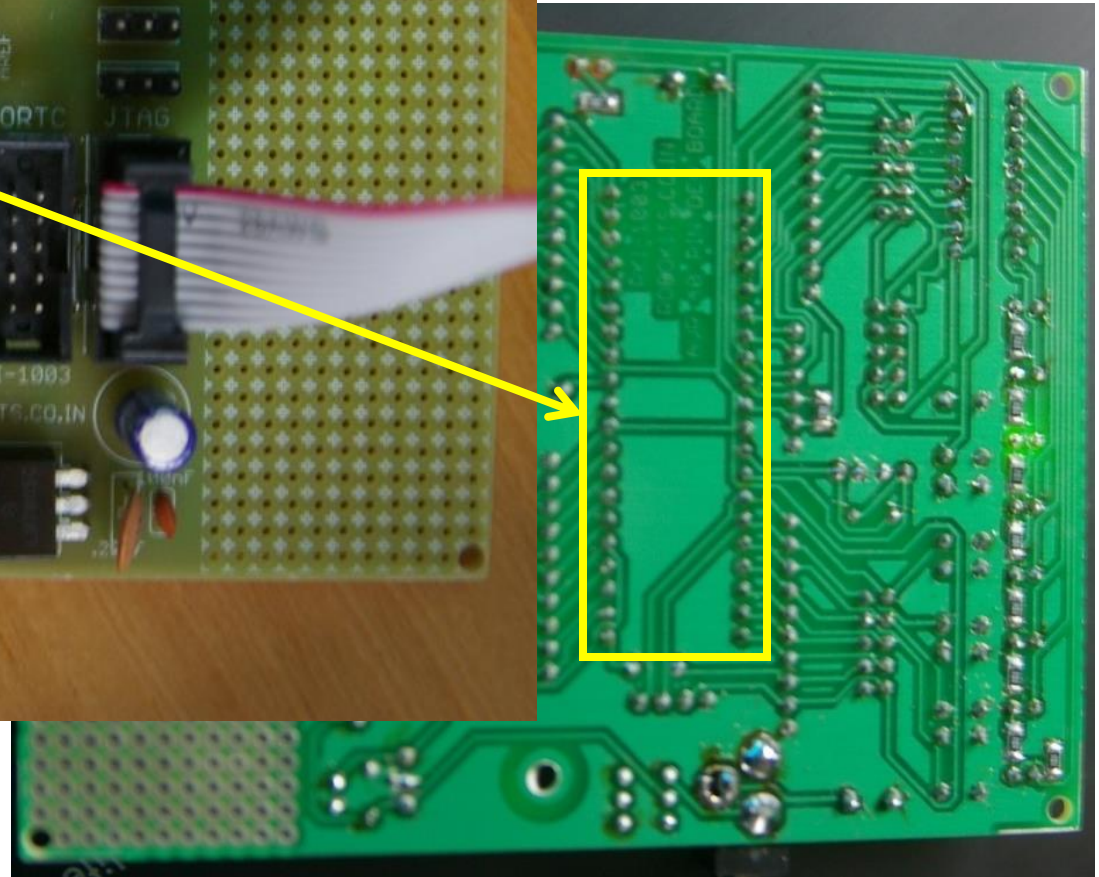
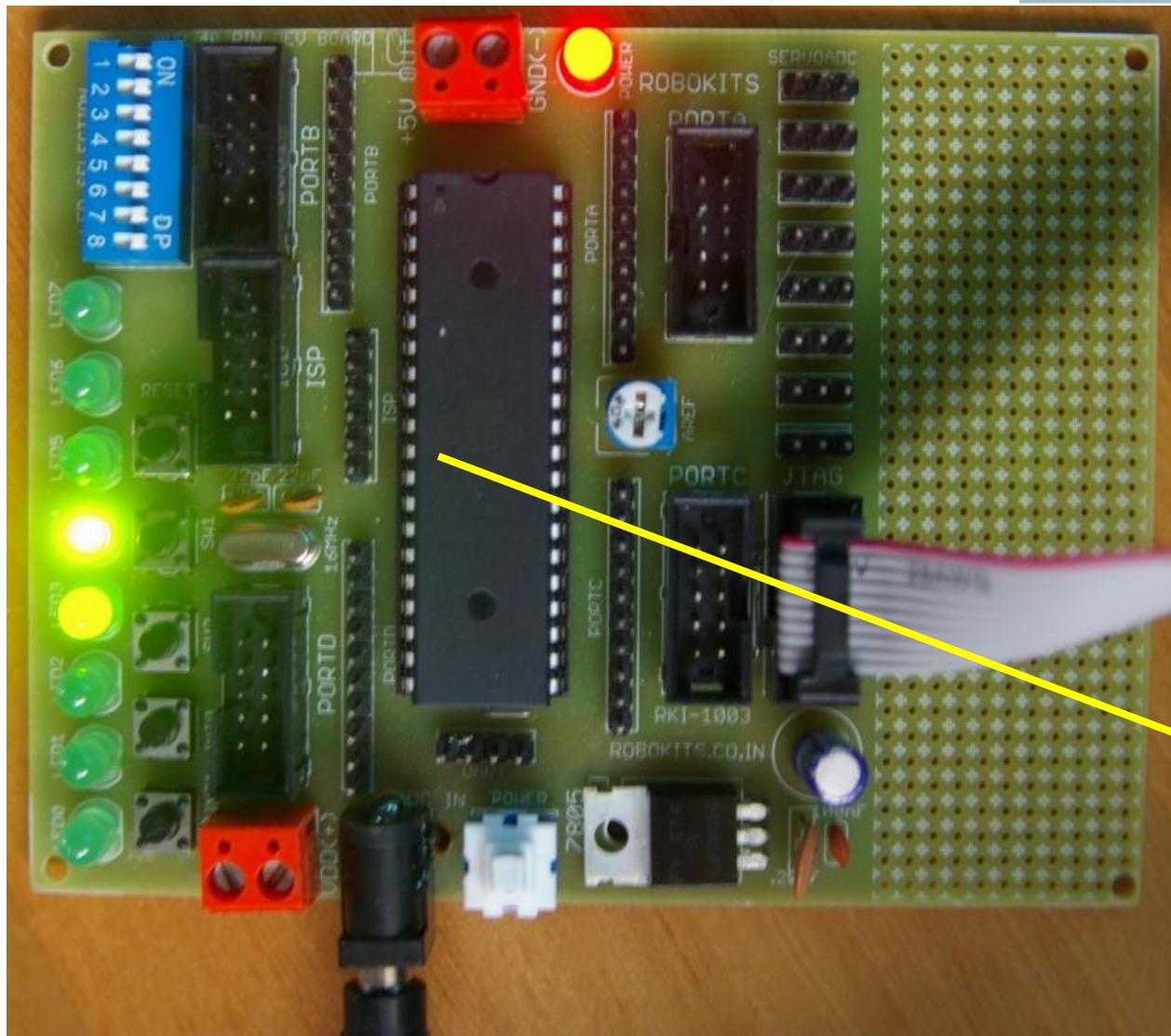
Ports A, B, C and D provide parallel I/O interfaces to outside world.



ATmega32 microcontroller Architecture, ...

■ Descriptions of the PORTS :

- **PA0...PA7: Port A** serves as the analog inputs to the A/D converter; it also used as an 8-bit bidirectional I/O port if the A/D converter is not used.
- **PB0...PB7: Port B** is an 8-bit bi-directional I/O port with internal pull-up resistors (selected for each bit); Port B also serves the functions of various special features of the ATmega32.
- **PC0...PC7: PORTC**, almost similar to PORTB - the Port C and B pins are tri-stated when a reset condition becomes active, even if the clock is not running.
- **PD0...PD7: Port D**; also similar to PORT B&C.



ATmega32 microcontroller Architecture, ...

- The device is manufactured using Atmel's high density nonvolatile memory technology.
- The On-chip ISP Flash allows the program memory to be reprogrammed in-system through an SPI, by a conventional nonvolatile memory programmer, or by an On-chip Boot program running on the AVR core.
- The boot program can use any interface to download the application program in the Application Flash memory.
- Software in the Boot Flash section will continue to run while the Application Flash section is updated, providing true Read-While-Write operation.

ATmega32 microcontroller Architecture, ...

■ **ATMega32 Programmer Model- Memory types:**

1. 2KB SRAM: used for temporary data storage, volatile memory - lost when power is shut; fast read/write.

2. 1KB EEPROM: used for persistent data storage, non-volatile memory contents are retained when power is off (non-volatile); it is also fast read but slow write, can write individual bytes.

3. 32KB Flash Program Memory: used to store program code, non-volatile memory, contents retained when power is off ; fast to read; slow to write, can only write entire “blocks” of memory at a time.

Assembly language Programming with ATmega32 Instruction Set

- To code in assembler, one should have some idea about the architecture of the target hardware.
- The AVR micro-controller appears to the programmer as a set of General Purpose Registers (GPRs: R0 to R31), Special Functions Registers (SFRs) that controls the peripherals, some data memory (2kbytes of SRAM for Atmega32).
- Assembly language programming involves moving data between GPRs, SFRs and RAM, and performing arithmetic and logical operations on the data.

Assembly language Programming with ATmega32 Instruction Set, ... cont'

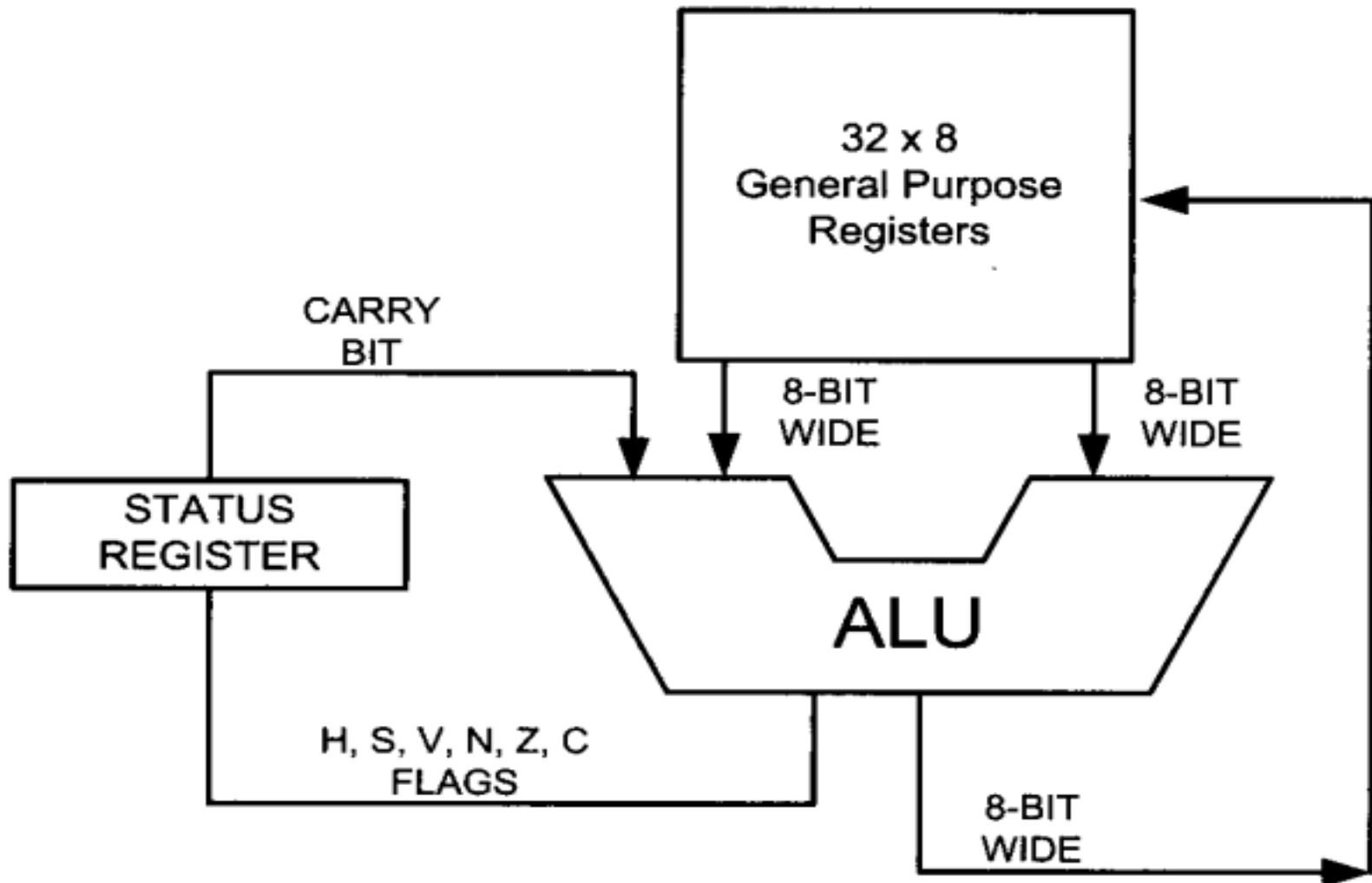
- To program in assembly language, we must understand the registers architecture of a given CPU and the role they play in processing data.
- With an 8-bit data type register, any data larger than 8 bits must be broken into 8-bit chunks before it is processed.
- The general purpose registers in AVR are the same as the **accumulator** in other microprocessor, they can be used by all arithmetic and logic instructions.

Assembly language Programming with ATmega32 Instruction Set, ... cont'

- There are 32 general purpose registers in AVR, labeled as R0 to R31.
- We cannot load constant values into registers **R0** to **R15** using LDI instruction. E.g, **LDI R5, 0x77** ;*invalid instruction*.
- When programming the GPRs of the AVR microcontroller with an immediate value, the following points should be noted:
 - To present a number in hex, put a dollar sign (\$) or a **0x** in front of it.
E.g; \$50 or 0x50
 - If values 0 to F are moved into an 8-bit register, the rest of the bits are assumed to be all zeros.
E.g; LDI R16, 0x5 the result will be R16=0x05; that is R16=00000101 in binary.
 - Moving value larger than 255 (FF) into GPRs will cause an error;
E.g, LDI R18, 0x7F2 ;illegal instruction because, \$7F2 > 8 bits .

Assembly language Programming with ATmega32 Instruction Set, ... cont'

- AVR General purpose registers as an accumulator for ALU:



AVR Status Register - similar to Flag register in x86 processor

- The status register is an 8-bit register, referred to as the flag register.
- The bits C, Z, N, S, V and H are called conditional flags, meaning that they indicate some conditions that result after an instruction is executed.

Bit	D7						D0	
SREG	I	T	H	S	V	N	Z	C
	C – Carry flag				S – Sign flag			
	Z – Zero flag				H – Half carry			
	N – Negative flag				T – Bit copy storage			
	V – Overflow flag				I – Global Interrupt Enable			

Review exercise

study the results and status flags generated by the following operations.

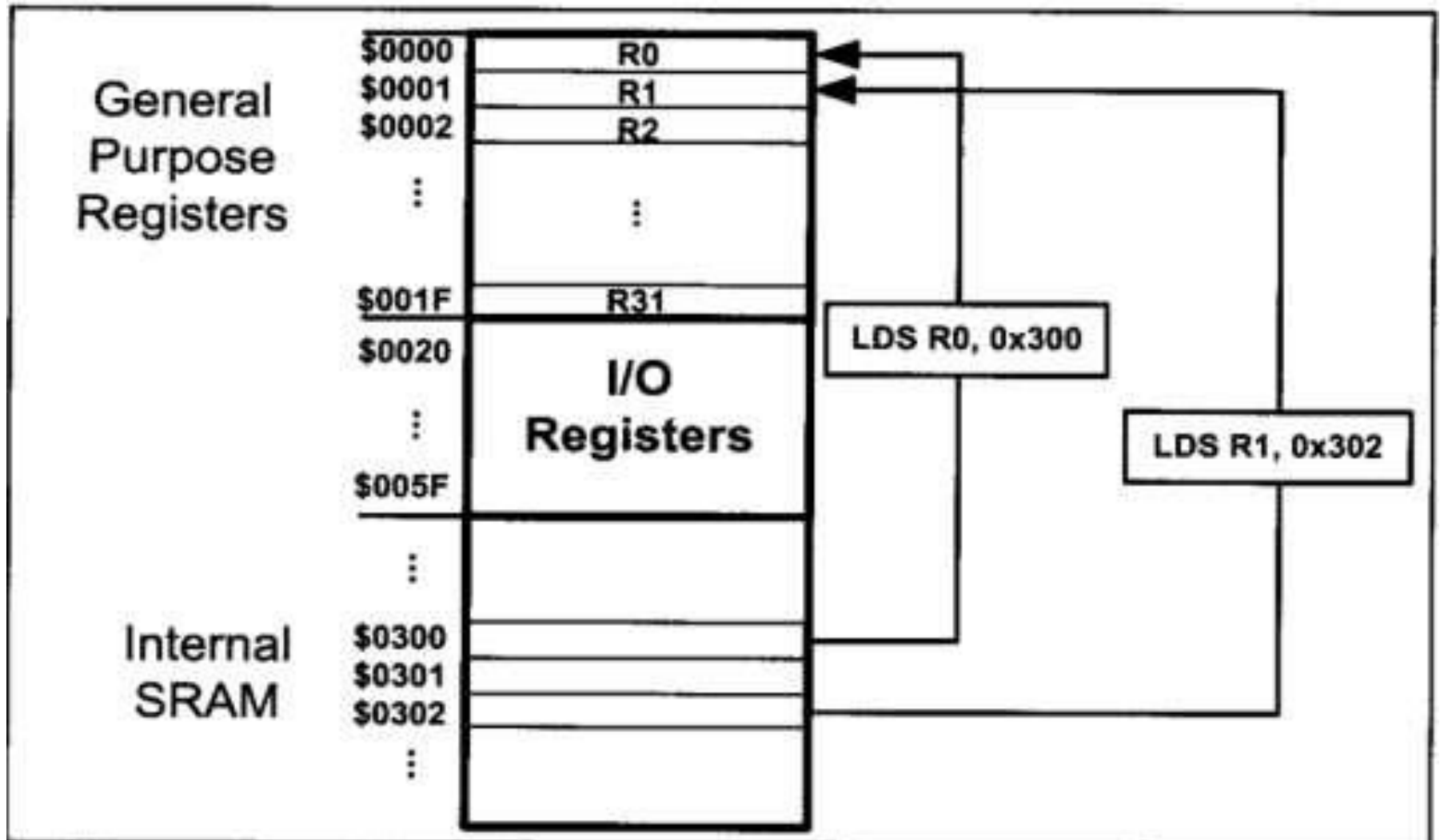
- LDI R16, 0x12
- LDI R17, 0x8
- NEG R16 ; 2's complement
- ADD R16, R17 ; summing
- LDS R18, SREG;
- STS PORTA, R18

Assembly language Programming with ATmega32 Instruction Set, ...cont'

- Any numerical value that needs to be used in the program needs to be first loaded into one of the GPRs of **R16 to R31**.
- E.g; If you want to load **0xff** into **DDRB** (data direction register to **PORTB**), you first need to load **0xff** into a **GPR** and then copy the content of the **GPR** into **DDRB**.
- This might seem like an unnecessary restriction to us who have been used to writing **DDRB=0xff** in **C**, but it is a necessary consequence of the streamlined hardware design of the processor which **C** hides from us.

Assembly language Programming with ATmega32 Instruction Set, ... cont'

General Purpose Registers: loading data from data memory



The code below (add.S) adds two numbers and displays the result on the LEDs connected to port B.

```
#include <avr/io.h>
.section .text ; denotes code section
.global main
main:
    LDI R16, 255 ; load R16 with 255
    STS DDRB, R16 ; set all bits of port B as output
    LDI R16, 2 ; load R16 with 2
    LDI R17, 4 ; load R17 with 4
    ADD R16, r17 ; R16 <- R16 + R17
    STS PORTB, R16 ; result to port B
    .END
```

Running this program lights LEDs D2 and D3

I/O Address Representation of Ports A, B, C and D(ATmega32)

- In the ATmega32, each port register has a **specific I/O address** used with instructions.
- In the table below, we can see I/O address representation for each port.

I/O Address Representation of ATmega32 Ports

Port	PINx (Input)	DDRx (Direction)	PORTx (Output)
PORTA	0x19	0x1A	0x1B
PORTB	0x16	0x17	0x18
PORTC	0x13	0x14	0x15
PORTD	0x10	0x11	0x12

Assembly language Programming with ATmega32 Instruction Set, ... cont'

Instruction set in assembly

- **LDI (Load Immediate)** : used to load a constant value to one of the registers R16-31 (that's a restriction. Load immediate can't work with **R0** to **R15**) .
- **OUT (output to any Special Function Register)**: The SFRs are mapped to the locations 0 to 3Fhex.
 - 0x17 and 0x18 are the I/O mapped addresses of the registers DDRB and PORTB respectively.
 - The SFRs are also mapped into the memory space to locations 20hex to 5Fhex.
 - Due to this reason you can use STS (Store Direct to SRAM) instruction instead of OUT but, to a different address.
 - OUT 0x17, R16 and STS 0x17, R16 achieves the same, but the former is compact.

Assembly language Programming with ATmega32 Instruction Set

Instruction set in assembly, ...

- Math
 - Addition
 - Subtraction
 - Multiply
 - Divide
- Comparisons
 - Register to register
- Data Transfer
 - Register to register
 - Data memory to register
 - SRAM and EEPROM
- Bit manipulation
 - Negate
 - 2's complement
 - Complement
 - 1's complement
 - Shift: left, right, circular
 - Set/Unset
 - Logical Operations
 - AND, OR, XOR
- Load/Store
 - Memory to register
 - Register to memory

Assembly language Programming with ATmega32 Instruction Set

Moving Data

- Register Direct: **MOV R1, R2** ; copies R2 to R1 . Two GPRs are involved in this. There are also operations that involves a single register like, **INC R1**
- **I/O Direct**: For moving data between the GPRs and the SFRs, since the SFRs can be accessed as I/O addresses.
E.g: OUT 0x17, R1; copies R1 to DDRB .
- **Data Direct**: The address of the memory location containing the data is specified, instead of the data itself. **E.g, LDS R1, 0x60** moves the content of memory location 0x60 to R1.

Assembly language Programming with ATmega32 Instruction Set

Moving Data ,... cont'

- **Data Indirect:** Here the address of the memory location is taken from the contents of the X, Y or Z registers.
 - X, Y and Z are 16 bit registers made by combining two 8 bit registers (X is R26 & R27; Y is R28 & R29; Z is R30 & R31; this is required for addressing memory above 255

LDI R26, 0x60 ; address of location 0x0060 to X

LDI R27, 0x00

LD R16, X ; load R16 with the content of memory location pointed to by X

The following example shows how to access a data variable using direct and indirect modes. (data-direct-var.S)

```
#include <avr/io.h>
.section .data           ; data section starts here
var1: .byte 0xEE         ; initialized global variable var1
.section .text           ; code section
.global __do_copy_data   ; initialize global variables
.global __do_clear_bss   ; and setup stack pointer
.global main
main:
    LDS R1, var1          ; load R1 using data direct mode
    STS DDRA, R1          ; display R1 on port A
    STS PORTA, R1
    LDI R26, lo8(var1)     ; load the lower and
    LDI R27, hi8(var1)     ; higher bytes of the address of var1 to X
    LD R16, X              ; Load R16 using data-indirect mode, data from where X is pointing to
    STS DDRB, R16         ; display R16 on port B
    STS PORTB, R16
    .END
```

Jumps and Calls:

- The execution flow of a particular program can be controlled by **CALL** and **JMP** instructions; (call-jump.S).
- As example below, the main program calls the subroutine in a loop, the data is incremented on each call.

```
#include <avr/io.h>
.section .text           ; code section starts
disp:                   ; our subroutine
    STS PORTB, R1        ; display R1 on port B
    INC R1               ; increments it
    RET                 ; and return
.global main
main:
    LDI, R16, 255
    STS DDRB, R16
    MOV R1, R16
loop:
    RCALL disp           ; relative call
    CALL disp           ; direct call
    RJMP loop
.end
```

Jumps and Calls: *JMP*, *RJMP* and *IJMP*

- In the AVR, there are three unconditional branches such as:
 - **JMP** – can go to any memory location in the 4M (word) address space of the AVR.
 - It is a 4-byte (32-bit) instruction in which 10 bits for opcode, and other 22 bits represent target address location (000000 to 3FFFFFF)
 - **RJMP** – relative jump which is 2-byte (16-bit) instruction, first 4 bits are the opcode and the rest (lower 12 bits) is the **relative** address of the target location (000 - \$FFF).
 - The address is divided into forward and backward jumps: if the jump is forward, then the relative address is positive, otherwise it is negative.
 - **IJMP** – indirect jump is a 2-byte (16-bit) instruction.
 - When the instruction executes, the PC is loaded with the contents of the Z register, so it jumps to the address pointed to by the Z register.
 - So, IJMP can jump within the lowest 64k words of the memory.

Assembly program and AVR memory status:

- **Example 1:** State the contents of R20, R21, and data memory location 0x120 after the execution of the following program.

```
LDI    R20, 5      ;load R20 with 5
LDI    R21, 2      ;load R21 with 2
ADD    R20, R21     ;add R21 to R20
ADD    R20, R21     ;add R21 to R20
STS    0x120, R20   ;store in location 0x120 the contents of R20
```

Assembly program and AVR memory status:

- **Example 1:** State the contents of R20, R21, and data memory location 0x120 after the execution of the following program.
- **Solution:**

The program loads R20 with value 5. Then it loads R21 with value 2. Then it adds the R21 register to R20 twice. At the end, it stores the result in location 0x120 of data memory.

Location	Data
R20	5
R21	
0x120	

After
LDI R20, 5

Location	Data
R20	5
R21	2
0x120	

After
LDI R21, 2

Location	Data
R20	7
R21	2
0x120	

After
ADD R20, R21

Location	Data
R20	9
R21	2
0x120	

After
ADD R20, R21

Location	Data
R20	9
R21	2
0x120	9

After
STS 0x120, R20

Programming in C to Interface peripherals, Interrupts, ISR and Timers

I/O programming in C

- All port registers of the AVR are both byte accessible and bit accessible.
 - To access a PORT register as byte, we use the PORTx label where x indicates the name of the port.
 - We access the data direction register in the same way, using DDRx to indicate the data direction of port x.
 - To access a PIN register as a byte, we use the PINx label where x indicates the name of the port.

Example: Write an AVR C program to get a byte of data from Port C. If it is less than 100, send it to Port B; otherwise, send it to Port D

- Solution:**

```
#include <avr/io.h>                                //standard AVR header
int main(void)
{
    DDRC = 0; _____ //Port C is input
    DDRB = 0xFF; _____ //Port B is output
    DDRD = 0xFF; _____ //Port D is output
    unsigned char temp;
    while(1)
    {
        temp = PINC; _____ //read from PIN C
        if ( temp < 100 )
            PORTB = temp;
        else
            PORTD = temp;
    }
    return 0;
}
```

I/O programming in C, ... cont'

- The I/O ports of ATmega32 are bit-accessible but some AVR C compilers do not support this feature; and
- Others do not have a standard way of using it.
- **For Example:** the code line below can be in *CodeVision* to set the first pin of Port B to one:
PORTB.0 = 1; But it cannot be used in other compilers such as **WinAVR**.

I/O programming in C, ... cont'

To write portable code that can be compiled on different compilers, we must use **masking**.

It is done by using **AND** and **OR** bitwise operations to access a single bit of a given register.

- So you can access a single bit without disturbing the rest of the byte.
- **Example:** A door sensor is connected to bit 1 of port B & an LED is connected to bit 7 of port C.
 - Write an AVR C program to monitor the door sensor and, when it opens, turn on the LED.

See next...

I/O programming in C, ... cont'

Solution:

```
#include <avr/io.h>                                //standard AVR header

int main(void)
{
    DDRB = DDRB & 0b11111101;                      //pin 1 of Port B is input
    DDRC = DDRC | 0b10000000;                       //pin 7 of Port C is output

    while(1)
    {
        if (PINB & 0b00000010)                     //check pin 1 (2nd pin) of PINB
            PORTC = PORTC | 0b10000000;             //set pin 7 (8th pin) of PORTC
        else
            PORTC = PORTC & 0b01111111;             //clear pin 7 (8th pin) of PORTC
    }
    return 0;
}
```

'o' means the port is in input status

'1' means the port is in output status

Programming in C to Interface peripherals, Interrupts, ISR and Timers, Cont'

What is an Interrupt?

- An interrupt is a signal (an “interrupt request”) generated by some event external to the CPU
- Causes CPU to stop currently executing code and jump to separate piece of code to deal with the event
 - Interrupt handling code often called an ISR (“Interrupt Service Routine”)
- When ISR is finished, execution returns to code running prior to interrupt

Programming in C to Interface peripherals, Interrupts, ISR and Timers, Cont'

Interrupt Overview

- Interrupt Sequence
 - Interrupt Event
 - Interrupt Request
 - Interrupt Service Routine
- Varying number of ISRs may be supported
 - Typically between 1-16 depending on platform
- Multiple events may be mapped to a single routine
 - i.e. any PORTA pin change
- Interrupt events may have priorities
- ISR is implemented inside a function with no parameters and no return value (void)
- Typically keep interrupt routines shorter than 15-20 lines of code

Programming in C to Interface peripherals, Interrupts, ISR and Timers, Cont'

- Main sources of interrupts are Hardware Interrupts and software interrupts.
- Hardware Interrupts
 - Commonly used to interact with external devices or peripherals
 - Microcontroller may have peripherals on chip
- Software Interrupts
 - Triggered by software commands, usually for special operating system tasks
 - i.e. switching between user and kernel space, handling exceptions
- Common hardware interrupt sources
 - Input pin change
 - Hardware timer overflow or compare-match
 - Peripherals for serial communication
 - UART, SPI, I2C – Rx data ready, tx ready, tx complete
 - ADC conversion complete
 - Watchdog timer timeout

Interrupt, ...

- Advantage of having interrupts is to avoid writing software code in such a way that the processor must frequently spend time checking the status of input pins or registers.
- **Coding Interrupts in AVR C:** in C language there is no instruction to manage the interrupts.
- So, in **WinAVR** the following have been added to manage the interrupts using C:
 - **Interrupt include file** – we should include the interrupt header file if we want to use interrupts in our program.
#include <avr\interrupt.h>
 - **cli() and sei()** – In assembly, the CLI and SEI instructions clear and set the I bit of the SREG register, respectively.
 - In WinAVR, the cli() and sei() macros do the same tasks.
 - **Defining ISR:** to write an ISR (interrupt service routine) for an interrupt we use the following structure: **ISR** (*interrupt vector name*) { *//our program* }; see table next for v-name;

See table below for interrupt vector name list.

Table 2.1 interrupts Vector Name for the ATmega32/ATmega16 in WinAVR

Interrupt	Vector Name in WinAVR
External Interrupt request 0	INT0_vect
External Interrupt request 1	INT1_vect
External Interrupt request 2	INT2_vect
Time/Counter2 Compare Match	TIMER2_COMP_vect
Time/Counter2 Overflow	TIMER2_OVF_vect
Time/Counter1 Capture Event	TIMER1_CAPT_vect
Time/Counter1 Compare Match A	TIMER1_COMPA_vect
Time/Counter1 Compare Match B	TIMER1_COMPB_vect
Time/Counter1 Overflow	TIMER1_OVF_vect
Time/Counter0 Compare Match	TIMER0_COMP_vect
Time/Counter0 Overflow	TIMER0_OVF_vect
SPI Transfer complete	SPI_STC_vect
USART, Receive complete	USART0_RX_vect
USART, Data Register Empty	USART0_UDRE_vect
USART, Transmit Complete	USART0_TX_vect
ADC Conversion complete	ADC_vect
EEPROM ready	EE_RDY_vect
Analog Comparator	ANALOG_COMP_vect
Two-wire Serial Interface	TWI_vect
Store Program Memory Ready	SPM_RDY_vect

Read more !

[Chapter 3: next](#)>>